



P0-Finalen 2021

Lösningförslag

Vaccin

Av: Leopold Hermansson och Axel Nilsson

Testfallsgrupp 1 ($Q=1$, $N \leq 100$)

Det är bara för en enda person P vi undrar när den får sitt vaccin.

Gå igenom alla dagar och håll koll på antal personer kvar tills P får sitt vaccin.

När antal personer kvar går ner till 0 har vi kommit till rätt dag, och skriver ut den.

Testfallsgrupp 2 ($Q \leq 100\ 000$, $N \leq 100\ 000$)

Här hinner vi inte svara för varje person i taget som i grupp 1, det skulle ta $O(QN)$ tid.

Istället skapar vi en lista $S[x]$ som är dagen personen på position x i kön får sitt vaccin.

Vi kan skapa listan genom att gå igenom alla dagar i ordning, och för varje dag j lägga till personerna som får vaccin den dagen till listan S genom att lägga till talet j k_j gånger. För att hitta svaret för en person bara tar man $S[p_i]$.

Tex för sample 1: $k = [1,3,5]$, $S = [1,2,2,2,3,3,3,3,3]$

Detta tar tid $O(\text{len}(S)) = O(15*N)$

Ekorren i trädet

Av: Fredrik Ekholm

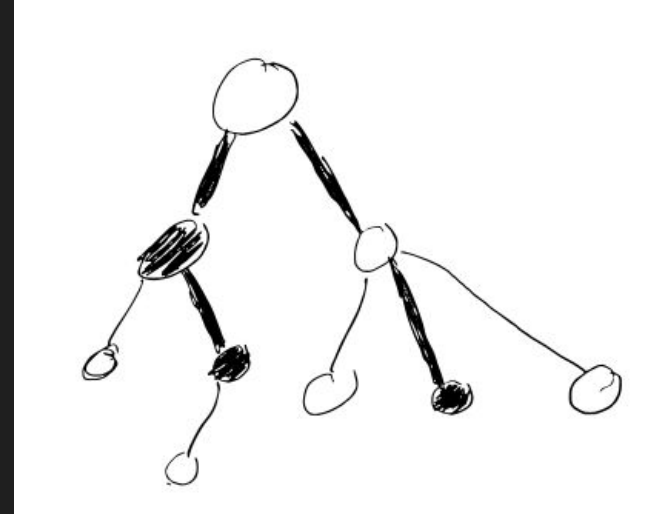
Första insikter

Ifall ett subträd inte innehåller någon nöt kan vi ignorera det.

Annars måste vi någon gång gå ner i subträdet, och sen gå ut från det. Då går vi längst kanten som leder ned i subträdet två gånger, vilket bidrar med 2 till totala avståndet.

Kalla alla kanter kanter som leder ned till subträd med nöt i för "viktiga".

Alltså är svaret $2 \times (\text{antal viktiga kanter})$



Testgrupp 2 ($K \leq N \leq 1000$)

Insikt: För varje nôt är alla kanter från roten till nôtén viktiga.

Vi kan börja att hitta föräldern till varje nod genom en traversering av trädets rekursiv DFS är nog enklast.

Sedan går vi igenom varje nôt och för varje nôt stegar vi upp till roten genom att iterativt gå till föräldern. Vi markerar alla kanter längs vägen som "viktiga".

Sist räknar vi hur många kanter som är viktiga.

$O(NK)$

Testgrupp 3 ($K \leq N \leq 100\,000$)

Vi kan räkna ut hela svaret rekursivt istället.

Vi har en funktion $\text{dfs}(V, P)$ som:

- lägger till alla viktiga kanter i V 's subträd till en global variabel
- returnerar en boolean som säger om det finns en nöt i V 's subträd.

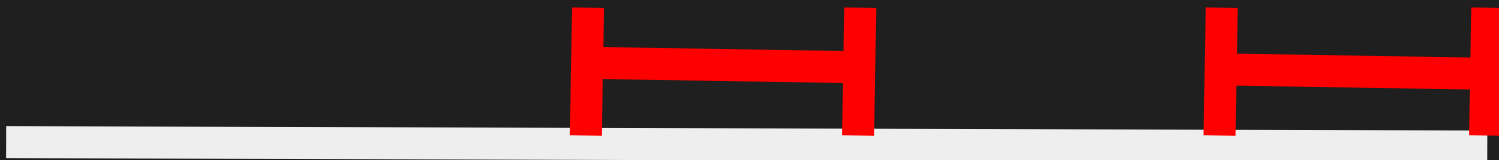
$\text{dfs}(V, P)$ kan räknas ut rekursivt genom att anropa alla grannar till V (som inte är P). V 's subträd har en nöt om några av barnens subträd har nöt eller om V har en nöt, och för varje barn är kanten till barnet viktig om barnets subträd har nöt.

Social distansering

Av: Fredrik Ekholm

Lösning...

Observation: Det är svårt att direkt räkna ut största avståndet, men det är lätt att testa om svaret är minst något tal X : Lägg personerna girigt så långt till vänster som möjligt!



Lösning...

Observation: Det är svårt att direkt räkna ut största avståndet, men det är lätt att testa om svaret är minst något tal X : Lägg personerna girigt så långt till vänster som möjligt!



Lösning för 76 poäng

- 1) Sortera listan med intervall.
- 2) Binärsök efter svaret X.
- 3) Gå igenom personerna och placera ut dem så långt vänster som möjligt.
- 4) Om du lyckades placera ut alla N personer är svaret $\geq X$, annars är det $< X$.

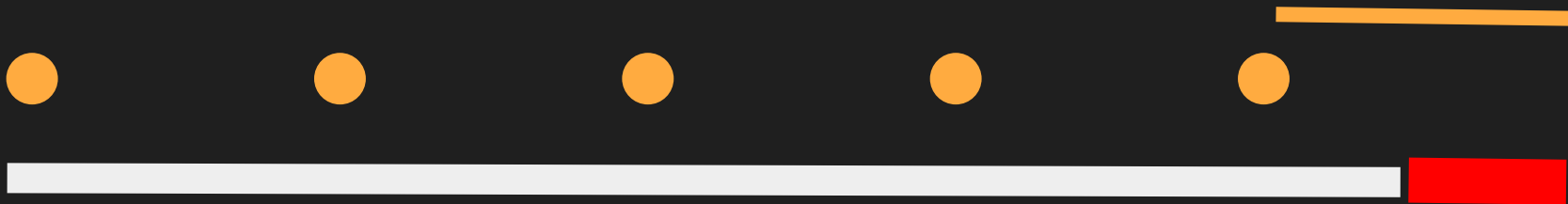
Komplexitet: $O((N+M)\log(K))$.



Lösning för 100 poäng

Om vi har ett segment med P platser, och vi testar om svaret är $\geq X$, så kan vi klämma in $\text{ceil}(P/X)$ personer där. Dessa personer kommer då blockera $(X - P \% X) \% X$ platser framför.

Vi kan då lösa problemet i $O(M \log K)$.



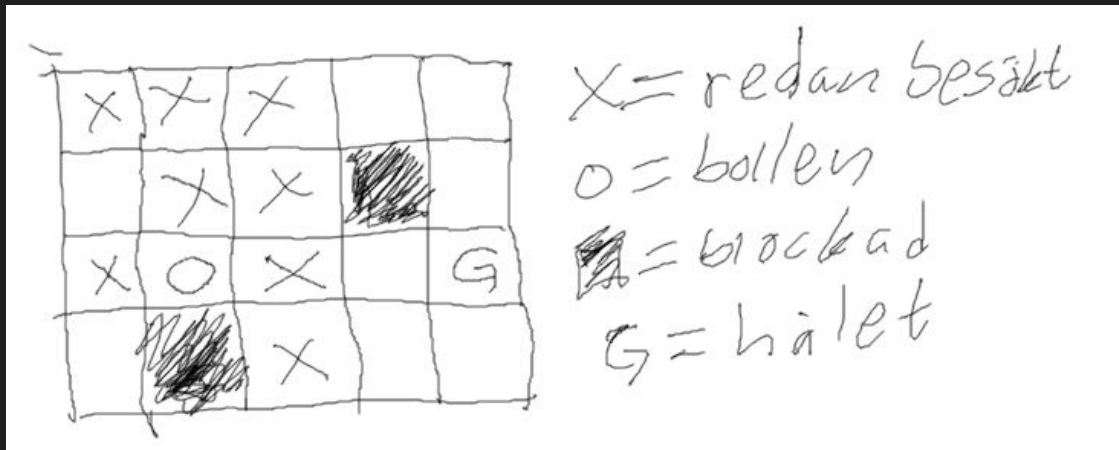
Minigolf

Av: Joakim Blikstad

Lösning

Kör BFS,

håll koll på rutorna du inte besökt
med hjälp av ett set för varje rad
och ett set för varje kolumn så
kan du snabbt



0 1 2 3 4 $K=3$

0	1	5	1	4	
1		1	4		
2	2	1	2	2	
3		1			

rows = 1

cols = 2

{4, 3, 1}

{0, 3, 4, 3, 1}

{~~4~~, ~~3~~, ~~1~~, 4, 3, 1}

{0, 2, 3, 4, 3}

x

]

$O(R \cdot C \cdot K)$ ~~Wrong~~ 226



Flyga drönare

Av: Ivar Källström

Testgrupp 1 ($N \leq 20$)

Här kan vi testa de 2^N olika delmängderna av batterier och se vilken delmängd (av de vi har råd med) som ger störst $E_{\text{tot}}/W_{\text{tot}}$.

$$O(N \cdot 2^N)$$

Testgrupp 2 ($w_i=0$)

Om inga batterier väger något vet vi redan att W_{tot} bara beror på drönarens vikt.

För att maximera $E_{\text{tot}}/W_{\text{tot}}$ räcker det nu att maximera E_{tot} .

Knapsack:

$dp[i][b]$ = (bästa summan av energier för de i första batterierna med budgeten b).

$= \max(dp[i-1][b - c_i] + e_i, dp[i-1][b])$

Testgrupp 3 (Alla batterier kostar lika mycket)

Antalet batterier vi ska ta är mindre än eller lika med B / c_0 , inga övriga begränsningar finns. Vi vill maximera $E_{\text{tot}} / W_{\text{tot}}$.

Testa om det finns batterier så att:

$$E_{\text{tot}} / W_{\text{tot}} > t \Leftrightarrow E_{\text{tot}} > t * W_{\text{tot}} \Leftrightarrow E_{\text{tot}} - W_{\text{batteri}} * t > t * W.$$

Maximera $E_{\text{tot}} - W_{\text{batteri}} * t$ genom att sortera batterierna i minskande ordning efter $e_i - w_i * t$ och ta de första k batterierna (eller avbryt om talen blir negativa).

Hitta största t , binärsökning.

$$O(\log(t_{\text{max}}) * N * \log(N))$$

Testgrupp 4

Kombinera lösning för grupp 2 och 3. Binärsök först efter L.

Knapsack:

$dp[i][b] = (\text{maximala } E_{\text{tot}} - W_{\text{tot}} * t \text{ för } i \text{ första batterierna med budgeten } b).$

$= \max(dp[i-1][b - c_i] + e_i - t * w_i, dp[i-1][b])$

Om $dp[N-1][B] \geq t * W$ är det möjligt att uppnå tiden t (samma omskrivning som förklarat tidigare)

Återupppfinna matematiken



$$\sum t_i \leq T$$

$$\text{maximera } \sum v_i$$



$$v_i - t_i$$

